

Collision and Preimage Resistance of the Centera Content Address

Robert Primmer*

Carl D'Halluin†

June 2005

Corrected edition: September 14, 2026

This corrected edition revises probability calculations, notation, and security qualifications in the June 2005 paper. Product descriptions remain historical; the numerical results are conditional models, not a certification of Centera.

Abstract

Centera uses cryptographic hash functions as a means of addressing stored objects, thus creating a new class of data storage referred to as CAS (content addressed storage). Such hashing serves the useful function of providing a means of uniquely identifying data and providing a global handle to that data, referred to as the Content Address or CA. However, such a model begs the question: how certain can one be that a given CA is indeed unique?

In this paper we describe fundamental concepts of cryptographic hash functions, such as collision resistance, preimage resistance, and second-preimage resistance. We then map these properties to the MD5 and SHA-256 hash algorithms, which are used to generate the Centera content address. Finally, we present conditional estimates of accidental collision probability for the Centera Content Address and distinguish them from adversarial security.

1 Introduction

EMC introduced Centera in April 2002, ushering in a new model of data storage referred to as Content Addressed Storage, or CAS [8]. The two principal ideas Centera introduced were a node-based structure referred to as RAIN, for redundant array of independent nodes, and an object store.

Centera presents a flat object store to the application. Applications submit data and are returned a global handle that uniquely refers to the data stored. The handle returned is a function of the data submitted. Upon submission, a 256-bit handle — called the Content Address, or CA — is calculated and returned to the application [20]. Nodes are divided into two groups: *Access Nodes*, which act principally as gateways between the cluster and the client application server; and *Storage Nodes*, which act principally as semi-autonomous storage containers. Data objects are referred to as *blobs*, and the associated metadata containers are referred to as *clips*.

The CA serves the twin role of unique data handle and MDC (Manipulation Detection Code). It is the hashing function that allows for the latter role. Since the CA is calculated based upon a bit representation of the content itself, a modification will generally change the hash-derived portion of the CA, subject to the possibility of a collision. Centera takes advantage of this property for achieving cache coherence and detecting when a file on disk has been damaged or in some way

*EMC² Corporation, Hopkinton, MA USA

†EMC² Corporation, Mechelen, Belgium

altered. Detail of this mechanism and the RAIN structure can be found in the Centera Product Description Guide [9].

In this paper we provide a general description of hash functions and their properties. We then focus on the MDx and SHA-x family of hash functions as the MD5 and SHA-256 schemes are currently employed by Centera for a portion of the CA calculation. We then describe the most common forms of attack on hash functions and relate those specifically to the Centera addressing schemes. Finally we derive conditional collision probabilities and explain the limits of adversarial security claims for the full CA.

2 Hash Functions

A hash function maps bit strings of an arbitrary finite length to fixed-length hash values. On the one hand a hash function must be fast to compute, but on the other it must also be collision-resistant, i.e. it must be computationally infeasible to find a collision (that is a pair of messages with the same hash value) [11]. Additionally, hash functions are intended to resist inversion (i.e. finding a message from a given hash value).

The basic idea of cryptographic hash functions is that a hash value serves as a compact representative image (sometimes called a *message digest*) of an input string, and can be used to uniquely identify that string. There are several types of hash functions [6][25]. The best known are:

- Hash functions based on block ciphers;
- Hash functions based on modular arithmetic;
- Hash functions based on cellular automata;
- Hash functions based on knapsack problem;
- Hash functions based on algebraic matrices; and
- Dedicated hash functions.

In this paper we focus on dedicated hash functions. Readers interested in a non-mathematical treatment of hash functions and cryptography are referred to [28] and [27] respectively.

2.1 Definitions

Perhaps the most widely known definitions for collision resistance, preimage-resistance, and second-preimage resistance come from the Handbook of Applied Cryptography (HAC) [2] which is based in part on the work of Preneel [5] which is the seminal work in this area, providing extensive treatment of cryptographic hash functions according to the information theoretic, complexity theoretic, and system based approaches. For our purposes we will rely primarily upon his descriptions.

The following terms are basic to cryptography in general and hash functions in particular.

Definition 1: A **one-way hash function** (OWHF) is a function h satisfying the following conditions:

1. The argument X can be of arbitrary length and the result $h(X)$ has a fixed length of n bits.
2. Given h and X , the computation of $h(X)$ must be “easy”.
3. *The hash function must be one-way in the sense that given a Y in the image of h , it is “hard” to find a message X such that $h(X) = Y$ (preimage resistant) and given X and $h(X)$ it is “hard” to find a message $X' \neq X$ such that $h(X') = h(X)$ (second-preimage resistant).*

Definition 2: A **collision resistant hash function** (CRHF) is a one-way hash function with the additional property that the hash function must be collision resistant: this means that it is “hard” to find two distinct messages that hash to the same result. The adversary must find a pair of distinct inputs X, X' satisfying $h(X) = h(X')$; both inputs are chosen by the adversary.

Note that “preimage resistant” is alternatively referred to as *one-way*; “(second) preimage resistant” is referred to as *weak collision resistant*; and “collision resistant” is referred to as *strong collision resistant*. Further mathematical rigor to the definition of these terms can be found in [2] and [17].

Definition 3: An n -bit hash function has **ideal security** if it satisfies the following conditions:

1. Given a hash output, producing a preimage or second preimage requires $\sim 2^n$ operations.
2. Producing a collision requires $\sim 2^{n/2}$ operations.

Definition 4: **Manipulation detection code** (MDC) is the umbrella term used to describe the class of hash functions that are unkeyed.

The tradeoff that must be considered when choosing a specific algorithm in this class is the tension between efficiency of computation versus degree of security. The two are often inversely related. Because the speed of a hash function is typically of critical importance, many block ciphers – while providing excellent security – are too slow to be practical. This gave rise to specialized hash functions, which are designed from scratch with speed as a principal consideration.

Definition 5: **Message authentication code** (MAC) is the umbrella term used to describe the class of hash functions that are keyed. While the introduction of a key adds state, it has the positive effect of allowing for expanded use of the hash function, with applications in areas such as digital signature authentication. For example, Centera uses a keyed MAC to validate operations between management station(s) and the Centera server.

A MAC is a function which takes the secret key k (shared between Alice and Bob) and the message m to return a tag $\text{MAC}_k(m)$. Here, Eve represents the adversary (eavesdropper). Eve sees a sequence $(m_1, a_1), (m_2, a_2), \dots, (m_q, a_q)$ of pairs of messages and their corresponding tags (i.e., $a_i = \text{MAC}_k(m_i)$) transmitted between the parties. Eve breaks the MAC if she can find a message m , not included among $m_1, m_2, \dots, m_q$, together with its corresponding valid authentication tag $a = \text{MAC}_k(m)$. Eve’s success probability is the probability that she breaks the MAC.

Hash functions are generally not keyed. However, there is interest in using cryptographically strong hash functions as the basis for MAC schemes. Bellare et al. [16] describe a means of using an iterative hash function as the basis for a MAC by keying the hash function via the initial vector, instead of the more common mechanism of inputting the key as part of the data hashed by the function.

2.2 Iterated Hash Functions

Most known hash functions (including MD5 and SHA) are based on a compression function with fixed size input; they process every message block in a similar way. The information is divided into t blocks X_1 through X_t . If the total number of bits is not a multiple of the block length, the information is padded to the required length (using a so-called *padding rule*). The hash function can then be described as follows:

$$\begin{aligned}
H_0 &= IV \\
H_i &= f(X_i, H_{i-1}) \quad i = 1, 2, \dots, t \\
h(X) &= g(H_t)
\end{aligned} \tag{2.1}$$

Where IV is the abbreviation for Initial Value (or Initial Vector); H_{i-1} serves as the n -bit chaining variable between stage $i - 1$ and stage i ; and the result of the hash function is denoted with $h(X)$. The function f is called the *round* or *compression* function, and the function g is called the *output transformation*. It is often omitted (that is, g is often the identity function); in the case of Centera, g maps bit strings to a base32-notation (digits 0-9 and A-V), which has no impact on the security of the hash. Two elements in this definition have an important influence on the security of a hash function: the choice of the padding rule and the choice of the IV . Figure 1 illustrates the mechanics of a typical iterated hash function [2].

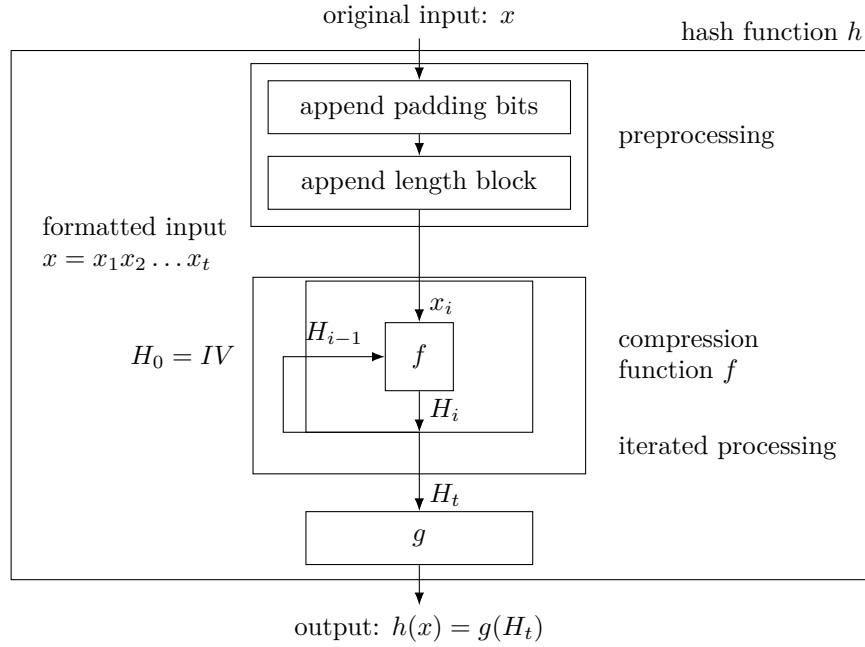


Figure 1: Iterated Hash Functions

It is recommended that the padding rule is unambiguous (i.e., there do not exist two messages that can be padded to the same padded message); at the end one should append the length of the message; and the IV should be defined as part of the description of the hash functions (this is called MD-strengthening, after Merkle and Damgård). In some cases one can deviate from this rule, but this will make the hash function less secure and may lead to trivial collisions or second preimages. Note that the function f , the IV , and the padding scheme are publicly known.

Research on iterated hash functions examines which properties of the compression function are inherited by the complete hash. Collision resistance and second-preimage resistance require separate reasoning. The discussion below distinguishes length-dependent second-preimage attacks from the collision-preservation theorem for a length-strengthened construction. For the latter result the output transformation g is the identity function; a non-injective truncation must be analyzed separately.

Compression-function security. The relationship between a compression function and the iterated hash depends on the attack model, message length, padding, and allowed choice of chaining values. The historical Lai–Massey discussion [29] is not used here to claim a universal 2^n second-preimage lower bound. In particular, long-message attacks [11] must also be considered.

Theorem (Merkle–Damgård, length-strengthened form). Let $f : \{0, 1\}^n \times \{0, 1\}^b \rightarrow \{0, 1\}^n$ be collision resistant. Encode each supported message injectively into blocks $X_1, \dots, X_t$, including padding and its length, so that no valid encoding is a proper suffix of another. With a fixed initial value, define

$$\begin{aligned} H_0 &= IV, \\ H_i &= f(H_{i-1}, X_i), \quad i = 1, \dots, t, \\ h(X) &= H_t. \end{aligned} \tag{2.2}$$

Here t counts *all* encoded blocks, including the length encoding. The order of the two arguments of f is state, then block in this formula; (2.1) uses block, then state. This consistent endpoint replaces the original recurrence’s undefined H_{t+1} .

To see why the construction preserves collision resistance, trace two equal final states backwards. Unequal compression inputs giving an equal output constitute a collision in f . Otherwise the last blocks and preceding states must coincide, and this reasoning repeats. Equal-length encodings would then be identical; unequal-length encodings would violate the suffix condition. Thus distinct messages with equal hashes yield a compression-function collision. Length fields must be unambiguous over the supported message lengths; merely adding arbitrary padding is insufficient [2, 15, 24].

Recent findings [11] show that for a typical n -bit iterated hash function, one can find a second preimage for a 2^k -block message in about $k \cdot 2^{n/2+1} + 2^{n-k+1}$ operations for long messages, assuming abundant memory. This is an attack-work estimate, not a lower bound on every possible attack.

3 MDx and SHA Functions

The MDx functions are a form of “Dedicated hash functions” defined earlier in §2. The MD4 [21] hash function was introduced by Ron Rivest in 1990. While weaknesses were discovered with MD4 [15][30][32], MD4’s significance is that it served as the starting point for the development of a series of similar iterated hash functions. MD4-derived hash functions include the 256-bit extension of MD4 [23] and the Secure Hash Algorithm (SHA) designed by NIST/NSA, later substituted by the slightly revised SHA-1 [10]. Additionally MD4 is optimized to be highly efficient on 32-bit word computers and is thus able to produce 128-bit hash results far quicker than alternative methods, e.g. via modified block ciphers.

To counter the weaknesses discovered with MD4 Rivest introduced MD5 [22] in 1991 to strengthen the hash function accordingly. MD5 differs from MD4 on the following points [3].

- A fourth round was added.
- The second round function has been changed from the majority function $(X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z)$ to the multiplexer function $(X \wedge Z) \vee (Y \wedge \neg Z)$.
- The order in which input words are accessed in rounds 2 and 3 was changed.
- The shift amounts in each round have been changed. None are the same now.
- Each step now has a unique additive constant.
- Each step now adds in the result of the previous step.

Where $\wedge$ denotes bit-wise AND, $\vee$ denotes bit-wise OR, $\oplus$ denotes bit-wise XOR, and $\neg Z$ denotes the bit-wise complement of Z .

3.1 MD5 Detail

MD5 is perhaps the most extensively covered dedicated hash function in the literature, likely due to its popularity. Rivest details the algorithm and provides sample C source in [22]. Schneier provides a survey of several of the more frequently used hash functions in [7]. For our purposes a subset of that information is sufficient.

The MD5 algorithm follows the iterative structure described in §2.2, where the hash is computed by repeated application of a compression (or *round*) function to successive blocks of the message. The message to be hashed is first padded to a multiple of 512 bits and then divided into a sequence of 512-bit message blocks. The compression function takes two inputs, a 128-bit chaining value and a 512-bit message block, and produces as output a new 128-bit chaining variable, which is input to the next iteration of the compression function [4]. In every step one of the chaining variables is updated. A typical step operation of the round function is

$$A \leftarrow B + \text{ROTL}_{32}(A + \Phi(B, C, D) + X[j] + K, s).$$

Each 512-bit block comprises 16 words $X[0], \dots, X[15]$ of 32 bits; j selects a word for the current step. All additions are modulo 2^{32} , and $\text{ROTL}_{32}(u, s)$ rotates a 32-bit word left by s bits. The Boolean function Φ and constant K depend on the round and step.

The initial register values, written as ordinary hexadecimal *32-bit integers*, are [22]

$$\begin{aligned} A &= 0x67452301, & B &= 0xefcdab89, \\ C &= 0x98badcfe, & D &= 0x10325476. \end{aligned} \tag{3.1}$$

RFC 1321 also lists their bytes in little-endian order, for example 01 23 45 67 for A . The original paper printed that byte sequence as if it were an integer; the distinction is now explicit.

There are four nonlinear functions, one used in each operation (a different one for each round).

$$\begin{aligned} \Phi_1(X, Y, Z) &= (X \wedge Y) \vee (\neg X \wedge Z) \\ \Phi_2(X, Y, Z) &= (X \wedge Z) \vee (Y \wedge \neg Z) \\ \Phi_3(X, Y, Z) &= X \oplus Y \oplus Z \\ \Phi_4(X, Y, Z) &= Y \oplus (X \vee \neg Z) \end{aligned} \tag{3.2}$$

These functions are designed so that if the corresponding bits of X , Y , and Z are independent and unbiased, then each bit of the result will also be independent and unbiased. The function Φ_1 is the bit-wise conditional: If X then Y else Z . The function Φ_2 is the bit-wise conditional: If Z then X else Y . The function Φ_3 is the bit-wise parity operator [7]. After the last message block has been processed, the final chaining value is output as the hash of the message.

3.2 SHA Functions

The Secure Hash Algorithm (SHA) [10] is a digest algorithm proposed by the US NIST (National Institute of Standards and Technology) agency as a standard digest algorithm in August 2002. NIST

released a first version of this algorithm — referred to as "SHA" back in 1992. It subsequently discovered a weakness with SHA and released a second version which it referred to as "SHA-1".

Where SHA-1 has a 160 bit hash value, NIST also defined hash functions with larger hash output such as SHA-224 (224 bits), SHA-256 (256 bits), SHA-384 (384 bits) and SHA-512 (512 bits). The cryptographic community generally considers SHA-256, SHA-384 and SHA-512 to be stronger than MD5.

Detail on the internals of the SHA family can be found in [10].

4 Hash Collisions

MD5 was widely deployed when the original paper was written. The collision attacks reported in 2004–2005 [30, 31] demonstrate why deployment and statistical behavior do not establish cryptographic collision resistance. RFC 6151 explicitly rejects MD5 for uses that require collision resistance [35]. The original statement that no practical full SHA-1 collisions were known was historical: a practical full SHA-1 collision was published in 2017 [34]. SHA-256 has a different design and output size; the absence of an attack in a historical survey is not a proof of security.

4.1 Practical Effects of Induced Collisions

Accidental collisions and deliberately constructed collisions require separate models. A small birthday probability for ordinary content says nothing about an adversary's ability to choose colliding inputs. The calculations below apply to distinct, non-adversarial contents whose hashes are modeled as independent uniform values. Repeated identical content is not a collision between distinct messages.

A collision in an address also does not, by itself, establish that an application can overwrite data: authorization, duplicate detection, and write-path semantics must be assessed separately.

4.2 Non-induced Collisions

For an ideal n -bit output, q independent trials against one fixed digest succeed with probability $1 - (1 - 2^{-n})^q$. The mean waiting time is 2^n trials, and the 50% point is approximately $(\ln 2)2^n$. For a fixed original message, finding a different message with that digest is a *second-preimage* problem. The original 2^{127} average for a 128-bit digest was incorrect under this model.

If any of r distinct target digests is acceptable, a fresh independent trial succeeds with probability $r/2^n$, so its mean waiting time is $2^n/r$. This generic multi-target model neither gives control over which file is matched nor proves an attack cost for real MD5 or a compound CA. Candidate trials can be evaluated offline; they need not be stored in a cluster. Any-pair collisions instead follow the birthday calculation in §5.1.

5 Collision Probabilities of Centera CA

Centera has three naming schemes, referred to as **M**, **GM** and **M++**. The **M** naming scheme returns a 128-bit CA that is simply the MD5 hash of the associated content. The **GM** naming scheme returns a 256-bit CA that is comprised of an MD5 hash of the associated content, a random bit-string, timestamp, counter and header bits. The **M++** naming scheme returns a 256-bit CA that is comprised of the MD5 and a truncated SHA-256 hash of the associated content. Detail for **M** is provided in §5.2; **GM** detail is provided in §5.4; **M++** is treated in §5.5. For now the salient

point is that the various naming schemes return CAs of different bit size (128 bits for **M** and 256 bits for **GM** and **M++**), and hence have differing collision probabilities and preimage resistance.

5.1 Birthday Paradox

For uniformly and independently distributed birthdays, 253 *other* people give approximately a 50% chance of matching a specified person's birthday (254 people total if that person is in the room):

$$1 - \left(\frac{364}{365}\right)^{253} \approx 0.50. \quad (5.1)$$

However, if you change the question to include one key difference, the answer changes dramatically.

The modified question is: how many people need to be present in a room for *any two* people to have $\approx 50\%$ chance of sharing the same birthday? Answer: 23. This number is surprisingly low, which leads to this phenomenon being given appellations such as “birthday surprise” or “birthday paradox.” The surprisingly small result for the second question becomes clear when you consider that with 23 people in the room there are 253 *pairs* of people in the room; i.e., $23 \cdot 22/2 = 253$. Of course, for this result to hold the people in the room must have an even (random) distribution of birthdays; clearly this result would not stand if the room was full of twins.

We now add some rigor and generalization to the birthday paradox.

Theorem (birthday upper bound): Let $P_{\text{collision}}(N, q)$ denote the probability to have at least one collision when we throw $q > 1$ balls at random into $N > q$ buckets, giving us equation (5.2) below.

$$P_{\text{collision}}(N, q) \leq 0.5 \frac{q(q-1)}{N} \quad (5.2)$$

The union bound below is useful for rare collisions. The exact probability and its exponential approximation are given after the proof, so that a bound is not mistaken for an equality. Additional birthday bounds and analysis are discussed in [26].

Proof: Let C_i be the event that the i -th ball is thrown in a non-empty bucket, i.e. it collides with one of the previous balls. Then $P[C_i]$ is at most $(i-1)/N$ since when the i -th ball is thrown, there are at most $i-1$ different occupied buckets and the i -th ball is equally likely to land in any of them. Now we have,

$$\begin{aligned} P_{\text{collision}}(N, q) &= P[C_1 \vee C_2 \vee \cdots \vee C_q] \\ &\leq P[C_1] + P[C_2] + \cdots + P[C_q] \\ &\leq \frac{0}{N} + \frac{1}{N} + \frac{2}{N} + \cdots + \frac{q-1}{N} \\ &\leq \frac{q(q-1)}{2N}. \end{aligned} \quad (5.3)$$

The bound in (5.2) is useful when collisions are rare, but it is not the probability itself. For $0 \leq q \leq N$ independent uniform samples,

$$P_{\text{collision}}(N, q) = 1 - \prod_{i=0}^{q-1} \left(1 - \frac{i}{N}\right) \approx 1 - \exp\left(-\frac{q(q-1)}{2N}\right).$$

The exponential approximation applies near the birthday scale for large N ; the linear approximation $q(q-1)/(2N)$ requires that this ratio be small. At $q \approx \sqrt{N}$, the probability is approximately $1 - e^{-1/2} = 39.35\%$. The 50% threshold is approximately $\sqrt{2N \ln 2} = 1.17741\sqrt{N}$. These statements assume independence and uniformity, not merely an even histogram of outputs.

5.2 Centera with the M naming scheme

For distinct, non-adversarial contents, assume MD5 outputs behave as independent uniform 128-bit values. Statistical tests motivate this model but do not prove it, and it does not apply to chosen collision attacks. Set $N = 2^{128}$. At $q = 2^{64}$ the collision probability is approximately 39.35%; the 50% threshold is approximately $1.17741 \cdot 2^{64}$. For $q = \alpha 2^{64}$, the union bound becomes

$$P_{\text{collision}}(2^{128}, q) \leq \min \left\{ 1, \frac{\alpha 2^{64} (\alpha 2^{64} - 1)}{2 \cdot 2^{128}} \right\}. \quad (5.4)$$

When $q \gg 1$ and $\alpha \ll 1$, the bound and the probability are both approximately $\alpha^2/2$.

Table 1 reports the rare-collision approximation to three significant figures. Capacities are decimal *logical payload* totals, not physical protected-storage requirements: 1 kB is 1,000 bytes.

Number of objects	Capacity with avg file size 10 bytes	Capacity with avg file size 1 kB	Approximate M collision probability
10^6	10 Megabyte	1 Gigabyte	1.47×10^{-27}
10^7	100 Megabyte	10 Gigabyte	1.47×10^{-25}
10^8	1 Gigabyte	100 Gigabyte	1.47×10^{-23}
10^9	10 Gigabyte	1 Terabyte	1.47×10^{-21}
10^{10}	100 Gigabyte	10 Terabyte	1.47×10^{-19}
10^{11}	1 Terabyte	100 Terabyte	1.47×10^{-17}
10^{12}	10 Terabyte	1 Petabyte	1.47×10^{-15}
10^{13}	100 Terabyte	10 Petabyte	1.47×10^{-13}
10^{14}	1 Petabyte	100 Petabyte	1.47×10^{-11}
10^{15}	10 Petabyte	1 Exabyte	1.47×10^{-9}

Table 1: Accidental collision probability for M; decimal logical payload capacities

At 10,000 files per second for 1,000 years of 365 days each, $q = 3.1536 \cdot 10^{14}$ and the modeled probability is approximately $1.46 \cdot 10^{-10}$. A total of 10^{15} files of 1 kB represents 1 EB of logical payload and takes about 3,171 years at that rate. Protection and metadata overhead require additional physical capacity.

For adversarial collisions, MD5’s cryptanalytic weaknesses replace this random-input model [30, 31]. Reusing a known collision pair is not the same task as constructing one for application-constrained content.

For a *given* file, finding a distinct file with the same M address is a second-preimage problem. Reference [11] gives a generic long-message attack with estimated work $k2^{65} + 2^{129-k}$ MD5 compression calls for a target near 2^k blocks. The historical “100Mb” limit is ambiguous and is not independently verified. Interpreting it as 100 MB (10^8 bytes) or 100 MiB ($100 \cdot 2^{20}$ bytes) gives fewer than 2^{21} padded 512-bit blocks; 100 megabits is smaller still. Substituting $k = 21$ gives roughly 2^{108} calls as a long-message reference scale, not a proved minimum for all files or attacks. Shorter targets have different work estimates.

5.3 Random Generators

In order to move towards the stronger GM naming scheme, we need the following definitions from the HAC [2].

Definition 6: A **pseudorandom bit generator** (PRBG) is a deterministic algorithm which, given a truly random binary sequence of length k , outputs a binary sequence of length $l \geq k$ which “appears” to be random. The input to the PRBG is called the *seed*, while the output of the PRBG is called a *pseudorandom bit sequence*.

Definition 7: A pseudorandom bit generator is said to pass the **next-bit test** if there is no polynomial-time algorithm which, on input of the first l bits of an output sequence s , can predict the $(l+1)^{st}$ bit of s with probability significantly greater than $\frac{1}{2}$.

Definition 8: A PRBG that passes the next-bit test is called a cryptographically secure pseudorandom bit generator (CSPRBG).

5.4 Centera with the GM naming scheme

Although the probabilities of having collisions with the M naming scheme are minimal, a second naming scheme, called GM, was introduced which includes additional random bits in the resultant CA. This additional randomness has the effect of further reducing the probability of collisions.

The Centera GM CA contains the following components:

- **M:** a 128 bit MD5 hash based on the file content;
- **G:** a 70 bit random string generated by a cryptographically secure pseudorandom bit generator;
- **T:** a 35 bit timestamp, with second resolution (actually, one unit of **T** is 1024 milliseconds);
- **C:** a 10 bit counter; and
- **H:** 13 bits used as “header” codes.

5.4.1 Timestamp and counter

Use $\Delta = 1,024$ milliseconds per timestamp tick and $C_0 = 2^{10} = 1,024$ counter values. A 35-bit timestamp wraps after $2^{35}\Delta$ milliseconds, about 1,116 years of 365 days. The following bound assumes coherent, monotone timestamp bins with no wrap, rollback, or reuse during the interval; otherwise objects with repeated timestamp values must be grouped together.

Let A be the number of access nodes and m an integer upper bound on writes assigned to any one timestamp by any one node. If S is expressed in files per second, using $m = \lceil 1.024S \rceil$ requires a bound on writes *within each tick*; a long-term average rate alone is insufficient. Each node’s counter advances by one per write and does not reset within a tick. A given counter value can then occur at most $\lceil m/C_0 \rceil$ times per node. Consequently, for any fixed (T, C) bin,

$$n_{T,C} \leq B, \quad B = A \left\lceil \frac{m}{C_0} \right\rceil. \quad (5.5)$$

This bound keeps the integer quantization and holds without requiring $S > A$. It does not depend on the nodes having independent counter offsets.

5.4.2 Applying the Birthday Paradox

Two distinct contents can share a full GM address only if M, G, T, C, and H all match. Ignoring H gives a conservative bound. For the accidental-content model, assume that each candidate pair in a (T, C) bin has probability 2^{-198} of matching both its 128-bit M and 70-bit G components. This requires independent uniform M outputs and fresh uniform G values independent of the content, node, counter, and timestamp. A CSPRBG definition alone does not prove these properties of an implementation. A union bound gives

$$P_{T,C} \leq \frac{\binom{n_{T,C}}{2}}{2^{198}} \leq \frac{B(B-1)}{2^{199}}. \quad (5.6)$$

No independence between collision events is needed for this union bound. For Z elapsed milliseconds, at most $L = \lceil Z/\Delta \rceil + 1$ timestamp bins intersect the interval, allowing for partial ticks at both ends. There are at most LC_0 candidate bins:

$$P_{GM} \leq \min \left\{ 1, \frac{LC_0 B(B-1)}{2^{199}} \right\}. \quad (5.7)$$

For repeated identical content, or adversarially arranged MD5 collisions, M contributes no independent collision protection. With fresh independent uniform G values, replace 2^{198} in the pair model by 2^{70} ; (5.7) then has denominator 2^{71} . The stronger 198-bit model must not be applied to those cases.

5.4.3 Collision Probability with GM

For the historical illustrative workload $A = 100$, $m = 10,240$ writes per tick per node (10,000 files/second under the per-tick assumption), and 1,000 years of 365 days, $Z = 31,536,000,000,000$ milliseconds. Thus $B = 1,000$ and $L = 30,796,875,001$, giving

$$P_{GM} \leq \frac{30,796,875,001 \cdot 1,024 \cdot 1,000 \cdot 999}{2^{199}} \approx 3.92 \cdot 10^{-41}. \quad (5.8)$$

This is a conditional upper bound, not impossibility and not an empirical measurement of Centera. It must not be compared directly with a per-bit disk error rate without specifying how many bits are read and over what interval. A content checksum can help detect corruption; a successful match is not an absolute guarantee of integrity.

For an attacker, fresh independent server-chosen G values give at most $\binom{u}{2}/2^{70}$ collision probability among u compatible writes in one (T, C, H) group whose M values have already been made equal. Matching a *fixed* target G in u independent compatible trials has probability $1 - (1 - 2^{-70})^u$. These are trial-count statements; converting them into elapsed years requires evidence about accepted write rates and timestamp control.

The archived 2005 correspondence reports that a raw-write SDK path can allow caller control of G. Therefore server-chosen randomness and exclusively online attacks cannot be asserted for every write path. If G is selectable, its random-collision factor provides no security against that caller; control of the other fields and the system's write checks must be analyzed. An address collision alone does not establish overwrite capability.

5.5 Centera with the M++ naming scheme

The historical M++ construction concatenates 128 MD5 bits, eight formatting bits, and 120 SHA-256 output bits. Its deterministic addressing supports identical-content naming. The original paper describes GM deduplication as a planned release feature; that release history is not established here.

For accidental collisions on distinct contents, assume the joint MD5 and truncated SHA-256 output is uniform over $N = 2^{248}$ values and independent between contents. This is a modeling assumption about the joint output, not a consequence of adding bit lengths. At $q = 2^{124}$ the birthday probability is approximately 39.35%, and the 50% threshold is approximately $1.17741 \cdot 2^{124}$. At 10,000 files per second for 1,000 years of 365 days, $q = 3.1536 \cdot 10^{14}$ gives approximately $1.10 \cdot 10^{-46}$, compared with $1.46 \cdot 10^{-10}$ for M under its random-input model.

For adversarial inputs, Joux’s multicollision construction shows that concatenated iterated hashes need not achieve the collision strength of an ideal hash with their combined output length [1]. This does not justify a general claim that every truncated concatenation has exactly the strength of its strongest component.

The original illustrative attack builds a 2^{60} -member MD5 multicollision family and searches for a collision in 120 SHA-256 output bits. It presumes that suitable equal-length MD5 collision blocks can be chained and that the resulting SHA-256 prefixes behave randomly. A naive scan of 2^{60} candidates of 120 message blocks costs about $120 \cdot 2^{60}$ SHA-256 compression calls, plus padding, MD5-family construction, and storage/search costs. This is a conditional attack estimate, not a lower bound on required work or a proof that future improvements cannot help. Its birthday success probability at that family size is about 39%, not certainty.

For second preimages, the original 2^{119} claim is not established by [11]. SHA-256 has a 256-bit chaining state even when only 120 output bits are retained; substituting 120 for its state width in that attack is invalid. Moreover, a second preimage of M++ must match both the MD5 and SHA-256 portions. A defensible concrete attack cost for that combined target is not derived here. The distinction between a fixed target and any of many targets in §4.2 also applies.

5.6 Summary of naming schemes

The introduction of the **GM** and **M++** naming scheme illustrates the flexibility of the Centera architecture, specifically designed to allow for seamlessly switching amongst naming schemes. Cryptography will continue to evolve, and due to the high importance in the commercial and military world, hash functions will be under continuous scrutiny. Therefore, it is important that Centera is able to adapt its content addressing schemes to reflect the state of the art in cryptography.

Table 2 summarizes the strength of the various naming schemes.

The second column gives the approximate 50% point under the stated accidental-content model. The attack columns deliberately distinguish conditional estimates from established bounds. GM results depend on the write path, randomness, and timestamp/counter behavior; no universal “impossible” classification or time-to-attack claim follows from this paper.

6 Conclusion

In this paper we have described basic principles of operation for iterative hash functions in general and MD5 in particular. We then described the naming schemes used within Centera and mapped the

	Approximate 50% accidental-collision point	Adversarial collision	Given-file second preimage
M	$1.17741 \cdot 2^{64}$ files	MD5 collision attacks apply	Length-dependent; historical 2^{108} reference scale only
GM	Depends on bin occupancy and runtime	Depends on write-path control and G generation	Depends on target-field access and write checks
M++	$1.17741 \cdot 2^{124}$ files	Conditional multicollision estimate; see §5.5	Concrete cost not established here

Table 2: Naming schemes: model-dependent probabilities and attack qualifications

properties described earlier to the Centera Content Address (CA). We demonstrated the likelihood of a CA collision in a Centera cluster. Finally, we concluded that hash functions will continue to evolve, and that the Centera system is architected to quickly adjust its naming scheme strategy to the latest insight in the relative strengths of cryptographic hash algorithms.

The corrected probability calculations require explicit randomness and workload assumptions. Small accidental-collision probabilities do not establish adversarial collision or second-preimage resistance. GM write-path behavior and a concrete M++ second-preimage analysis remain unresolved.

7 Acknowledgements

The authors would like to acknowledge Prof. Bart Preneel for his extensive help in writing this paper, and David Black for his help in deriving the proper function for computing collision probabilities when using the GM naming scheme.

We would also like to thank reviewers of early drafts of this paper for their time and useful comments. Specifically: Dirk Nerinckx, Jiri Schindler, Geert Denys, and Kristof Van Belleghem.

8 References

- [1] A. Joux, *Multicollisions in iterated hash functions. Application to cascaded constructions*, Advances in Cryptology – Crypto’04, Lecture Notes in Computer Science 3152, M. Franklin, Ed., Springer-Verlag, 2004.
- [2] A. Menezes, P. van Oorschot, S. Vanstone. *Handbook of Applied Cryptography*, CRC Press, 1997.
- [3] B. den Boer, A. Bosselaers. *Collisions for the compression function of MD5*, Advances in Cryptology – Eurocrypt ’93, Lecture Notes in Computer Science, vol. 773, Springer-Verlag, 1994, pages 293 – 304.
- [4] B. Kaliski, M. Robshaw. *Message Authentication with MD5*, CryptoBytes, Vol 1, Number 1, Spring 1995.
- [5] B. Preneel. *Analysis and Design of Cryptographic Hash Functions*, PhD Thesis, Katholieke Universiteit Leuven Belgium, 1993.
- [6] B. Preneel. *Cryptographic Primitives for Information Authentication – State of the Art*, State of the Art in Applied Cryptography 1997, LNCS 1528, Springer-Verlag, 1998, pages 49 – 104.

- [7] B. Schneier. *Applied Cryptography: Protocols, Algorithms, and Source Code in C*, 2nd Edition, Wiley & Sons, Inc., 1996.
- [8] EMC². *Centera Data Sheet*, s.a., http://www.emc.com/pdf/products/centera/centera_ds.pdf
- [9] EMC². *Centera Product Description Guide*, s.a., http://www.emc.com/pdf/products/centera/centera_guide.pdf
- [10] National Institute of Standards and Technology (NIST), *FIPS Publication 180-2, Secure Hash Standard (SHS)*, August 2002
<http://csrc.nist.gov/publications/fips/fips180-2/fips180-2.pdf>
- [11] J. Kelsey, B. Schneier. *Second Preimages on n -bit Hash Functions for Much Less than 2^n Work*, s.a., <http://eprint.iacr.org/2004/304.pdf>
- [12] H. Dobbertin. *The Status of MD5 After a Recent Attack*, CryptoBytes, Vol 2, Number 2, pages 1 – 6, 1996.
- [13] H. Dobbertin. *RIPEMD with two-round compress function is not collision-free*, Journal of Cryptography, s.a.
- [14] H. Handschuh, H. Gilbert. *Security Level of Cryptography – SHA-256*, Proceedings of SAC 2003, s.a.
- [15] I.B. Damgård. *A design principle for hash functions*, Advances in Cryptology: Proceedings of Crypto '89, volume 435 of Lecture Notes in Computer Science, pages. 416 – 427. Springer-Verlag, New York, 1990.
- [16] M. Bellare, R. Canetti, H. Krawczyk. *Keying Hash Functions for Message Authentication*, Advances in Cryptology – Crypt '96 Proceedings, LNCS Vol 1109, Springer-Verlag, 1996
- [17] P. Rogaway, T. Shrimpton. *Cryptographic Hash-Function Basics: Definitions, Implications, and Separations for Preimage Resistance, Second-Preimage Resistance, and Collision Resistance*, Lecture Notes in Computer Science, Springer-Verlag, 2004.
- [18] R. Canetti, D. Micciancio, O. Reingold. *Perfectly One-Way Probabilistic Hash Functions*, Proceedings of STOC'98, pages 131 – 140, ACM Press, s.a.
- [19] RIPE. *Integrity Primitives for Secure Information Systems, Final Report of RACE Integrity Primitives Evaluation (RIPE-RACE 1040)*, LNCS 1007, A. Bosselaers, B. Preneel, Eds., Springer-Verlag, 1995.
- [20] R. Primmer. *Efficient Long-Term Data Storage utilizing Object Abstraction with Content Addressing*, August 2003. http://powerlink.emc.com/HighFreq/P_PL_h1050_efficient_long_term_data_storage_utilizing_content_addressing_ldv.pdf
- [21] R.L Rivest. *RFC 1320: The MD4 Message-Digest Algorithm*, IETF, 1992.
- [22] R.L Rivest. *RFC 1321: The MD5 Message-Digest Algorithm*, IETF, 1992.
- [23] R.L. Rivest. *The MD4 Message Digest Algorithm*, Advances in Cryptology – Crypto '90, Lecture Notes in Computer Science, volume 537, pages 303 – 311, Springer-Verlag, 1991.
- [24] R. Merkle. *One-way hash functions and DES*, Advances in Cryptology: Proceedings of Crypto '89, volume 435 of Lecture Notes in Computer Science, pages 428 – 446. Springer-Verlag, New York, 1990.

- [25] S. Bakhtiari, R. Safavi-Naini, J. Pieprzyk. *Cryptographic Hash Functions: A Survey*, Center for Computer Security Research, Department of Computer Science, University of Wollongong.
- [26] S. Goldwasser, M. Bellare. *Lecture Notes in Cryptography*, MIT and UCSD, August 2001, pages 242 – 243.
- [27] S. Singh. *The Code Book – The Science of Secrecy From Ancient Egypt to Quantum Cryptography*, Anchor Books, September 2000.
- [28] S. Friedl. *An illustrated Guide to Cryptographic Hashes*, s.a., <http://www.unixwiz.net/techtips/iguide-crypto-hashes.html>.
- [29] X. Lai, J.L. Massey. *Hash functions based on block ciphers*, Advances in Cryptology, Proceeding Eurocrypt '92, LNCS 658, Springer-Verlag, 1993, pages 55-70.
- [30] X. Wang et al. *Collisions for Hash Functions MD4, MD5, HAVAL-128 and RIPEMD*, August 2004, Crypto 2004 Rump Session, <http://eprint.iacr.org/2004/199.pdf>
- [31] X. Wang et al. *How to Break MD5 and Other Hash Functions*, s.a., <http://www.infosec.sdu.edu.cn/paper/md5-attack.pdf>
- [32] X. Wang et al. *Cryptanalysis of the Hash Functions MD4 and RIPEMD*, s.a., <http://www.infosec.sdu.edu.cn/paper/md4-ripemd-attck.pdf>
- [33] X. Wang et al. *Collision Search Attacks on SHA-1*, February 13, 2005, <http://theory.csail.mit.edu/~yiqun/shanote.pdf>
- [34] M. Stevens, E. Bursztein, P. Karpman, A. Albertini, and Y. Markov. *The first collision for full SHA-1*, 2017. <https://ir.cwi.nl/pub/25624>
- [35] S. Turner and L. Chen. *Updated Security Considerations for the MD5 Message-Digest and the HMAC-MD5 Algorithms*, RFC 6151, March 2011. <https://www.rfc-editor.org/rfc/rfc6151.html>